

DAS 839 - NoSQL Systems

Assignment II

C Chandrahas Reddy – IMT2023037
Sarthak Maheshwari – IMT2023014
Harsh Mohta – IMT2023106

April 13, 2026

1 Introduction

This assignment explores large-scale text processing using the MapReduce programming model in Apache Hadoop, applied to a Wikipedia dataset. The goal is to implement and evaluate two approaches for building a co-occurrence word matrix — the **Pairs** approach and the **Stripes** approach — and analyze the effect of local aggregation techniques on runtime performance.

The work is organized into two main problems:

- **Problem 1:** Identifying the top 50 most frequent words from the corpus, then using these words to build a co-occurrence matrix for window sizes $d \in \{1, 2, 3, 4\}$ using both the Pairs and Stripes approaches. Additionally, two local aggregation strategies — map-function level and map-class level — are implemented and compared against the baseline for both approaches.
- **Problem 2:** Computing the Document Frequency (DF) for every term in the corpus using stemming and stopwords filtering, extracting the top-100 DF terms, and then computing a TF-DF relevance score for each term-document pair using the Stripes approach.

All jobs were executed on a corpus of **10,000 documents**. Key results include a best speedup of **20.04%** from in-mapper aggregation on stripes, 536,676 DF terms from Problem 2A, and 612,083 scored rows from Problem 2B.

2 Common Processing Pipeline

All parts follow the same preprocessing steps:

- Lowercase normalization and regex-based tokenization.
- Stopword filtering via Hadoop distributed cache.
- Reducer-side aggregation for consistent final counts.

In addition, **Problem 2** applies Porter stemming to normalize tokens before filtering — so words like “running” and “run” are treated as the same term. This reduces vocabulary size and improves the accuracy of document frequency counts. Problem 1 does not use stemming as the focus is on raw word co-occurrence among the top-50 frequent terms.

The corpus contains **10,000 documents**, each processed as a single input record.

3 Problem 1A: Top-50 Frequent Terms

Goal: Find the 50 most frequent words after removing stopwords.

How it works: Each mapper reads the stopwords list once from cache, then emits $(word, 1)$ for every valid token. A combiner sums counts locally before the shuffle. The reducer keeps a min-heap of size 50 to track the top terms efficiently.

Pseudocode

```
Mapper.setup:
  stopWords <- read cache file

Mapper.map(line):
  for token in tokenize(lowercase(line)):
    if token not in stopWords:
      emit(token, 1)

Combiner.reduce(term, counts):
  emit(term, sum(counts))

Reducer.reduce(term, counts):
  total <- sum(counts)
  push (term, total) into minHeap of size 50

Reducer.cleanup:
  output heap in descending order
```

Output Sample

apos	273057	quot	267968
s	206869	http	79094
1	75348	2	65386
category	60521	this	58742
0	54283	www	54131

Complexity: $O(T)$ for mapping, $O(V \log 50)$ for reducer heap maintenance, where T is total tokens and V is vocabulary size.

4 Problem 1B: Co-occurrence Using Pairs (Baseline)

Goal: Count how often each pair of top words appears near each other, for window $d \in \{1, 2, 3, 4\}$.

How it works: For each focus word at position i , the mapper looks at all neighbors within distance d and emits the pair $((w_i, w_j), 1)$. The reducer sums these counts. This is simple but generates a large number of intermediate records since every pair is emitted separately.

Pseudocode

```
Mapper.map(tokens):
  for i in [0..n-1]:
    if tokens[i] not in topWords: continue
```

```

    for j in [max(0,i-d)..min(n-1,i+d)]:
        if i != j and tokens[j] in topWords:
            emit((tokens[i], tokens[j]), 1)

Reducer.reduce(pair, counts):
    emit(pair, sum(counts))

```

Output

Pair output row counts: 2407, 2470, 2492, 2498 for $d = 1, 2, 3, 4$.

```

1 (0,0) 5572
2 (0,1) 3383
3 (0,10) 503
4 (0,11) 375
5 (0,12) 479
6 (0,2) 2430
7 (0,2006) 99
8 (0,2007) 66
9 (0,2008) 52
10 (0,2009) 56
11 (0,2010) 156
12 (0,2011) 44
13 (0,2012) 7
14 (0,3) 1776
15 (0,4) 1320
16 (0,5) 1343
17 (0,6) 1174
18 (0,7) 913
19 (0,8) 898
20 (0,9) 786

```

Figure 1: First 20 rows of co-occurrence pairs output for $d = 1$

Complexity: $O(L \cdot d)$ per record, roughly linear in total tokens for small d .

5 Problem 1C: Co-occurrence Using Stripes (Baseline)

Goal: Same as 1B but using a stripe (map) instead of individual pairs.

How it works: Instead of emitting one pair at a time, the mapper collects all neighbors of a focus word into a single map (stripe) and emits it as one record. The reducer merges all stripes for the same word. This reduces the number of emitted records compared to pairs, which is why stripes generally have lower shuffle overhead.

Pairs vs Stripes: Pairs emit one record per neighbor, while stripes group all neighbors into one record per focus word. This means stripes send less data through the shuffle phase, making them more efficient for large vocabularies. However, each stripe value is larger in size, which can increase serialization cost.

Pseudocode

```

Mapper.map(tokens):
    for i in tokens:
        if wi not in topWords: continue
        stripe <- empty map
        for j in neighborhood(i, d):
            if tokens[j] in topWords:
                stripe[tokens[j]] += 1
        emit(wi, stripe)

Reducer.reduce(wi, stripes):
    merged <- empty map

```

```

for stripe in stripes:
    for (neighbor, c) in stripe:
        merged[neighbor] += c
emit(wi, merged)

```

Output

Output has exactly **50 focus-word rows** for each d .

```

1 0 {d=36, i=6, time=15, m=187, n=385, p=95, state=5, 26amp=7, s=108, people=137, county=4, other=45, united=14,
two=7, http=107, category=13, world=11, war=2, quot=254, first=7, american=1895, 0=5572, 1=3383, city=16, 2=2438,
3=1776, history=28, 4=1320, 5=1343, 6=1174, 7=913, 8=898, uk=40, 9=786, this=40, 10=503, 2006=99, 11=375, 2008=52,
12=479, university=4, 2007=66, 2009=56, apos=105, www=3, states=13, years=24, 2011=44, 2010=156, used=2}
2 1 {d=99, i=249, time=84, m=327, n=751, p=599, state=22, 26amp=36, s=287, people=326, county=110, other=36, united=25,
two=35, http=230, category=53, world=29, war=23, quot=728, first=35, 0=3383, american=318, 1=4324, city=39, 2=3890,
3=2123, history=42, 4=1778, 5=1736, 6=1430, 7=1113, 8=1087, uk=757, 9=966, this=81, 10=747, 2006=304, 11=474, 12=464,
2008=353, university=19, 2007=331, 2009=335, apos=302, www=9, states=21, years=51, 2011=349, 2010=584, used=3}
3 10 {d=22, i=45, time=21, m=79, n=120, p=144, state=7, 26amp=1, s=119, people=456, county=9, united=12, other=14,
two=13, http=277, category=13, world=13, war=3, quot=245, first=29, 0=503, american=29, 1=747, city=9, 2=589, 3=521,
history=13, 4=486, 5=614, 6=487, 7=435, 8=472, uk=33, 9=524, this=22, 2006=474, 10=776, 11=534, 2008=786, 12=471,
university=10, 2007=889, 2009=947, apos=78, www=1, states=14, years=287, 2011=949, 2010=950, used=4}
4 11 {d=16, i=10, time=12, m=31, n=84, p=141, state=3, 26amp=4, s=34, people=47, county=4, other=17, united=6, two=15,
http=253, category=1, world=8, war=4, quot=142, first=10, 0=375, american=32, 1=474, city=11, 2=387, 3=387,
history=9, 4=328, 5=397, 6=254, 7=232, 8=275, uk=29, 9=532, this=16, 10=534, 2006=583, 11=432, 2008=754, 12=466,
university=9, 2007=882, 2009=652, apos=69, www=4, states=10, years=52, 2011=1051, 2010=1104, used=1}
5 12 {d=17, i=19, time=10, m=44, n=80, p=126, state=5, s=68, people=40, county=10, other=18, united=8, two=26,
http=164, category=6, world=5, war=5, quot=287, first=16, 0=479, american=18, 1=464, city=3, 2=448, 3=326,
history=10, 4=310, 5=365, 6=294, 7=335, 8=231, uk=26, 9=378, this=21, 2006=534, 10=471, 11=466, 2008=710, 12=500,
university=17, 2007=746, 2009=655, apos=60, www=2, states=14, years=117, 2011=1084, 2010=612, used=2}
6 2 {d=158, i=188, time=26, m=389, n=460, p=456, state=19, 26amp=16, s=285, people=252, county=66, other=36, united=39,
two=30, http=253, category=118, world=52, war=32, quot=1120, first=52, 0=2430, american=118, 1=3890, city=37, 2=2508,
3=2358, history=31, 4=1641, 5=1500, 6=1170, 7=878, uk=484, 8=911, 9=723, this=109, 2006=170, 10=589, 11=387,
2008=230, 12=448, university=19, 2007=223, 2009=285, apos=253, www=6, states=18, years=148, 2011=292, 2010=340,
used=7}
7 2006 {d=83, i=21, time=20, m=34, n=15, p=365, state=19, s=108, people=12, county=5, united=22, other=15, two=18,
http=696, category=53, world=56, war=4, quot=339, first=11, 0=99, american=24, 1=304, city=12, 2=178, 3=170,
history=43, 4=128, 5=150, 6=153, 7=161, uk=39, 8=128, 9=150, this=59, 2006=162, 10=474, 11=583, 12=534,
university=48, 2008=106, 2007=232, 2009=88, apos=123, www=2, states=18, years=6, 2011=49, 2010=96, used=1}
8 2007 {d=109, i=38, war=10, l=22, quot=408, time=26, m=34, first=17, n=14, p=331, american=37, state=12, 0=66,
1=331, city=16, 2=223, 3=201, history=73, s=85, 4=194, 5=210, 6=229, 7=149, 8=159, uk=16, 9=197, people=15, this=99,
10=889, 2006=232, 11=882, 12=746, 2008=295, county=11, university=67, 2007=228, united=21, other=54, two=15,
2009=107, apos=92, www=9, http=927, states=10, category=55, years=6, 2011=71, 2010=81}
9 2008 {d=82, i=12, time=16, m=28, n=16, p=380, state=14, s=69, people=17, county=18, united=35, other=39, two=24,
http=930, category=36, world=51, war=9, quot=431, first=13, 0=52, american=32, 1=353, city=18, 2=230, 3=192,
history=66, 4=239, 5=208, 6=201, 7=174, 8=221, uk=67, 9=187, this=108, 10=786, 2006=106, 11=754, 12=710, 2008=314,
university=61, 2007=295, 2009=250, apos=115, www=14, states=6, years=8, 2011=52, 2010=115, used=3}
10 2009 {d=81, i=16, time=22, m=18, n=18, p=182, state=21, 26amp=3, s=61, people=21, county=10, other=37, united=35,
two=22, http=995, category=35, world=65, war=5, quot=335, first=16, 0=56, american=26, 1=335, city=34, 2=285, 3=236,
history=48, 4=194, 5=203, 6=191, 7=169, 8=141, uk=74, 9=266, this=108, 10=947, 2006=80, 11=652, 12=655, 2008=250,
university=45, 2007=107, 2009=246, apos=100, www=21, states=10, years=8, 2011=74, 2010=273}

```

Figure 2: First 10 rows of co-occurrence stripes output for $d = 1$

Difference in Output Format: Pairs (1B) vs Stripes (1C)

The two approaches produce structurally different outputs:

- **Pairs (1B):** Each line represents a single co-occurring word pair and its count. For example:

```

(word1, word2)    42
(word1, word3)    17

```

Every neighbor of a focus word gets its own separate output row, so the number of rows grows with the window size and vocabulary overlap.

- **Stripes (1C):** Each line represents a focus word and a map of *all* its neighbors with their counts. For example:

```

word1    {word2=42, word3=17, word4=9, ...}

```

All neighbors are grouped into one record per focus word, so there are always exactly 50 output rows — one per top word — regardless of window size.

Both approaches capture the same co-occurrence information, but stripes compress it into fewer, larger records while pairs expand it into many smaller ones.

6 Problem 1E: In-Mapper Aggregation Variants

Goal: Test whether aggregating inside the mapper (before shuffle) speeds things up for both pairs and stripes.

Two aggregation levels were implemented for both pairs and stripes:

Variant	Aggregation Scope	When it Emits
Map-class-level pairs	Whole mapper lifetime	Once in <code>cleanup()</code>
Map-function-level pairs	Per <code>map()</code> call	End of each <code>map()</code>
Map-class-level stripes	Whole mapper lifetime	Once in <code>cleanup()</code>
Map-function-level stripes	Per <code>map()</code> call	End of each <code>map()</code>

Class-level vs Function-level: Class-level aggregation accumulates counts across *all* records a mapper processes and emits only once at the end. This gives maximum reduction in shuffle data. Function-level aggregation only aggregates within a single input record (one line), so it reduces duplicates per line but still emits after each line. Class-level is therefore more aggressive and generally more effective.

Generic Pseudocode

```
Mapper.setup:
  agg <- empty local hash structure

Mapper.map(record):
  update agg from record
  if function-level:
    emit agg; clear agg

Mapper.cleanup:
  if class-level:
    emit agg
```

Correctness Check (Pairs)

All pair variants produce bitwise-identical output to the 1B baseline for all window sizes — confirmed via SHA-256 checksums.

d	1B vs Map-Class	1B vs Map-Function	Result
1	true	true	pass
2	true	true	pass
3	true	true	pass
4	true	true	pass

For stripes, raw checksums differ due to non-deterministic map serialization order, but all variants produce the same 50 focus-word rows with consistent neighbor counts.

7 Runtime Analysis (Problem 1)

Variant	$d = 1$	$d = 2$	$d = 3$	$d = 4$	Avg
1B Pairs (baseline)	110	109	111	110	110.00
1C Stripes (baseline)	140	133	139	132	136.00
1E Map-Class Pairs	125	117	128	121	122.75
1E Map-Function Pairs	123	120	123	122	122.00
1E Map-Class Stripes	133	111	110	110	116.00
1E Map-Function Stripes	107	108	107	113	108.75

Key Observations

- **Pairs with in-mapper aggregation were slower** than the baseline (11.59% and 10.91% slower for class and function level). This is because the pairs key space is already small (only top-50 words), so the HashMap overhead adds cost without enough shuffle savings to compensate.
- **Stripes with in-mapper aggregation were faster** (14.71% and 20.04% faster). Since each stripe value is a map, aggregating them before emit reduces the number and size of shuffled records significantly.
- **Map-function-level stripes performed best overall** (avg 108.75s), slightly outperforming class-level stripes (116.00s). This may be because function-level aggregation avoids holding a large HashMap in memory for the full mapper lifetime.
- **Stripes baseline (136s) was slower than pairs baseline (110s)**, but after aggregation stripes become the fastest approach — showing that stripes benefit more from optimization.

8 Problem 2A: Document Frequency with Stemming

Goal: For every term in the corpus, count how many distinct documents it appears in (Document Frequency / DF).

How it works: Each mapper tracks which stemmed, non-stopword terms appear in its document using a set (so each term is only counted once per document). At cleanup, it emits $(term, docId)$ for each unique term. The reducer counts how many distinct document IDs each term received.

Porter stemming is applied so that words like "running" and "run" are treated as the same term, giving more accurate frequency counts.

Pseudocode

```
Mapper.setup:
  stopWords <- load and stem stopword list
  seenTerms <- empty set

Mapper.map(line):
  for token in tokenize(lowercase(line)):
    stem <- porterStem(token)
    if stem not in stopWords:
      seenTerms.add(stem)
```

```
Mapper.cleanup:
    for term in seenTerms:
        emit(term, docId)

Reducer.reduce(term, docIds):
    emit(term, countDistinct(docIds))
```

Output

Total DF terms: **536,676** | Top-100 list: **100 rows** | Runtime: **154.15 seconds**

```
1 9836
2 apo 9518
3 categori 9381
4 refer 9348
5 quot 8674
6 other 8415
7 1 8335
8 link 8294
9 thi 8190
10 2 8102
11 extern 8008
12 two 7949
13 us 7912
14 includ 7885
15 year 7795
16 first 7689
17 3 7519
18 http 7418
19 fr 7375
20 time 7349
21 state 7299
22 4 7025
23 5 7020
24 www 6780
25 see 6761
26 histori 6653
27 peopl 6625
28 0 6598
29 10 6597
30 6 6557
```

Figure 3: Sample rows from the top-100 DF output file

Complexity: $O(T)$ map phase; $O(U_d)$ memory per mapper where U_d is unique terms per document; $O(P)$ reduce phase where P is total emitted pairs.

9 Problem 2B: TF-DF Scoring

Goal: For each document, compute a relevance score for each of the top-100 terms using:

$$\text{score}(t, d) = tf(t, d) \cdot \ln\left(\frac{10000}{df(t) + 1}\right)$$

How it works: The top-100 DF file from 2A is loaded into memory via distributed cache. The mapper counts how many times each top-100 term appears in the current document (term frequency / TF) and emits a stripe. A combiner merges stripes locally before shuffle. The reducer computes the final score using the cached DF values.

Why stripes here? Since we only have 100 terms, each stripe is small and fits easily in memory. The combiner can efficiently merge these stripes locally, reducing shuffle data significantly — making stripes a natural fit for this task.

Pseudocode

- Stopword removal further trims the vocabulary, keeping the output focused on meaningful terms rather than common grammatical words.
- Loading stopwords via distributed cache means each mapper reads the file locally without any HDFS overhead, making setup fast even across many nodes.

Problem 2B – TF-DF Scoring:

- Restricting scoring to only the top-100 DF terms significantly reduces computation – instead of processing hundreds of thousands of terms, each mapper only tracks 100, keeping memory usage very low.
- The stripes approach is a natural fit here since the stripe size is bounded to at most 100 entries, making each emitted value compact and easy to merge.
- The combiner merges partial stripes before the shuffle phase, reducing the amount of data sent to the reducer and speeding up the job – this is why 2B (137.26s) completed faster than 2A (154.15s) despite producing more output rows (612,083 vs 536,676 – wait, 2A produced more DF terms but 2B scored more rows).
- Caching the top-100 DF file via `addCacheFile` means both the mapper and reducer can look up DF values instantly from a local in-memory HashMap, avoiding any extra HDFS reads during job execution.
- The scoring formula $tf \times \ln(10000/df + 1)$ naturally gives higher scores to terms that appear frequently in a document but rarely across the corpus, making the scores a useful measure of term importance per document.

10 Implementation Summary

Part	Summary
1A	Top-50 word frequency with combiner and reducer min-heap.
1B	Baseline co-occurrence pairs for $d = 1, 2, 3, 4$.
1C	Baseline stripe co-occurrence with reducer merge.
1E	Four in-mapper aggregation variants (pairs/stripes $\times$ class/function level).
2A	Full-corpus DF with Porter stemming, stopwords filtering, top-100 extraction.
2B	TF-DF scoring for top-100 terms using stripe mapper, combiner, and cached DF.

11 Conclusion

All parts were implemented and validated. The main takeaway from Problem 1 is that in-mapper aggregation helps stripes significantly but slightly hurts pairs — because stripes have larger shuffle payloads that benefit more from early merging. The best performing variant was map-function-level stripes at an average of 108.75 seconds. Problem 2 scaled well to the full corpus, with 2B completing faster than 2A despite producing more output rows, thanks to the combiner reducing shuffle load.